

STRUCTURA IDE

[Versión 1]

Un entorno de desarrollo modular y flexible, diseñado para optimizar la programación con Script Code Lite. Ofrece una interfaz sencilla y fácil de utilizar.

Creadores:

Aurora Del Ángel Martínez.

Nicohol González Santiago.

Olga Lidia Cervantes Del Ángel.

Román Narciso Arguelles Santiago.

Reynaldo Iván Antonio Ponce.

Tabla de contenido

Interfaz de usuario	3
1. Archivos.....	4
1.1 Abrir	5
1.2 Guardar.....	6
1.3 Guarda Como.....	6
1.4 Salir	7
2 Editar	7
3.- Compilar.....	8
4.- Ayuda	9
5._ Ejemplo de compilación	10

Interfaz de usuario

La interfaz del usuario o el menú principal del sistema contiene todas las opciones disponibles para ser utilizadas, por él usuario.

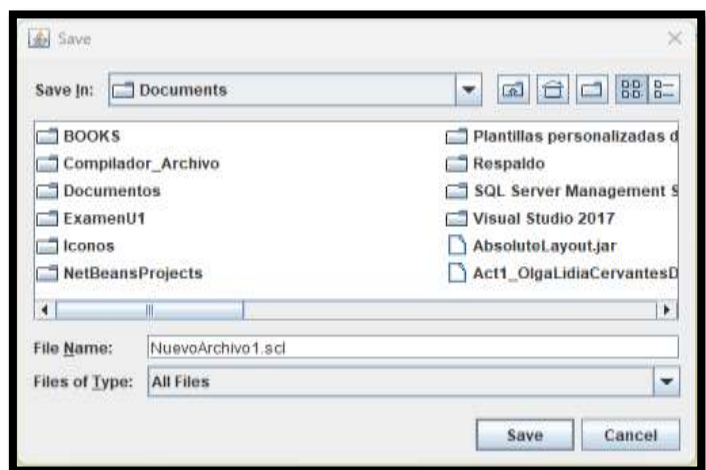
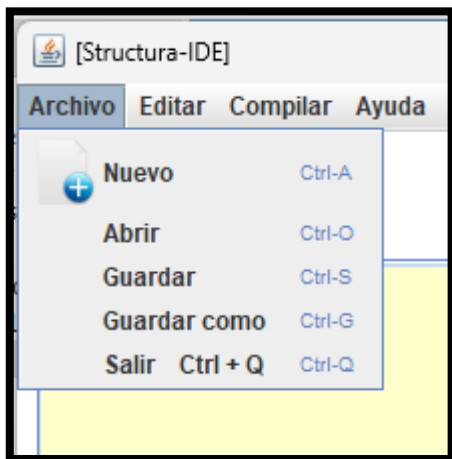


La interfaz cuenta con los siguientes submenús:

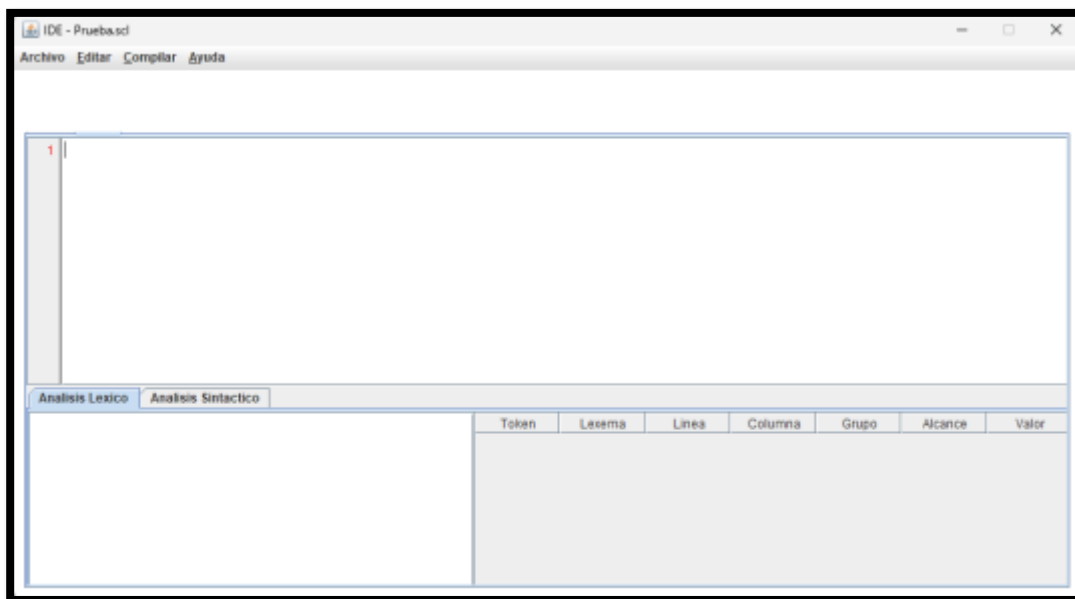
Archivo	Editar	Compilar	Ayuda
Nuevo (Ctrl+N)	Deshacer (Ctrl+Z)	Ejecutar (Ctrl+F6)	Manual de usuario (F1)
Abrir (Ctrl+O)	Rehacer (Ctrl +R)		Acerca de
Guardar (Ctrl+S)	Copiar (Ctrl+C)		
Guardar Como (Ctrl+G)	Cortar (Ctrl+X)		
Salir (Ctrl+Q)	Pegar (Ctrl+V)		

1. Archivos

Permite crear un nuevo archivo. Al hacer clic, se abrirá una ventana para seleccionar la carpeta de destino y asignar un nombre al archivo. También puede accederse a esta opción mediante el atajo de teclado **Ctrl + A**.



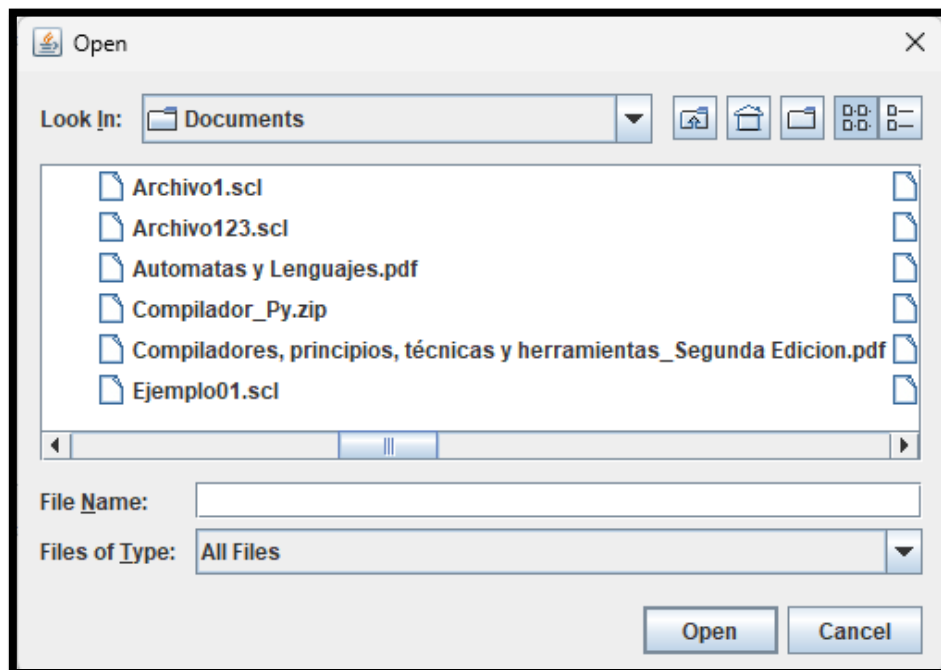
Una vez guardado el nuevo archivo, se habilitará la ventana de edición de código.



1.1 Abrir

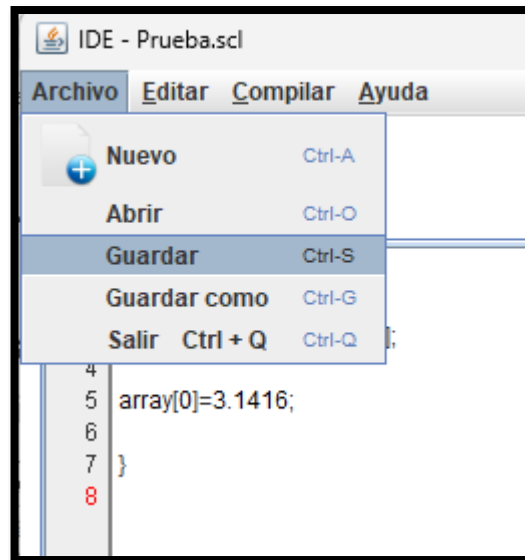
Permite abrir archivos previamente guardados. Solo se admiten archivos con la extensión especificada por el IDE; si se intenta abrir un archivo con otra extensión, no se podrá completar la acción.

La siguiente pestaña nos permite elegir el archivo que necesite abrir.



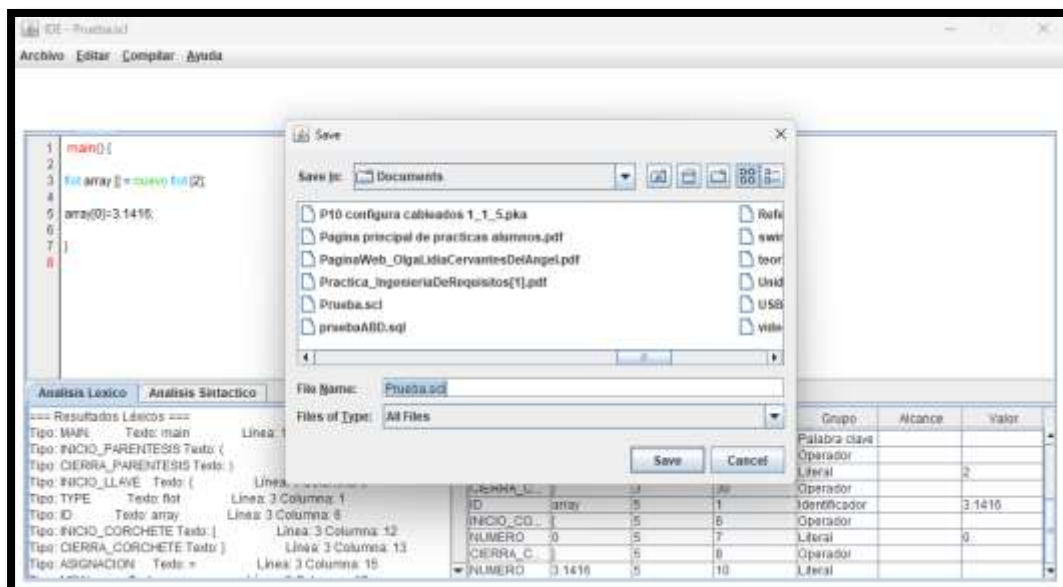
1.2 Guardar

Guarda los cambios realizados en el archivo actual. Es recomendable utilizar esta opción frecuentemente para no perder el progreso.



1.3 Guarda Como

Permite guardar una copia del archivo actual en una ubicación diferente y/o con un nombre distinto, según las preferencias del usuario.



1.4 Salir

Cierra el IDE. Es importante guardar los cambios antes de salir, ya que los datos no guardados se perderán.

2 Editar

2.1 Deshacer

Revierte la última acción realizada, como la eliminación de texto seleccionado.

2.2 Rehacer

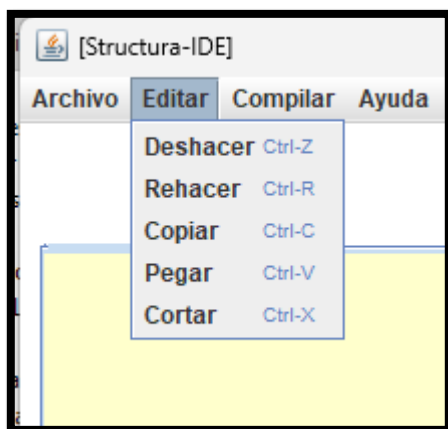
Restaura una acción que fue deshecha previamente.

2.3 Copiar

Copia el texto seleccionado al portapapeles.

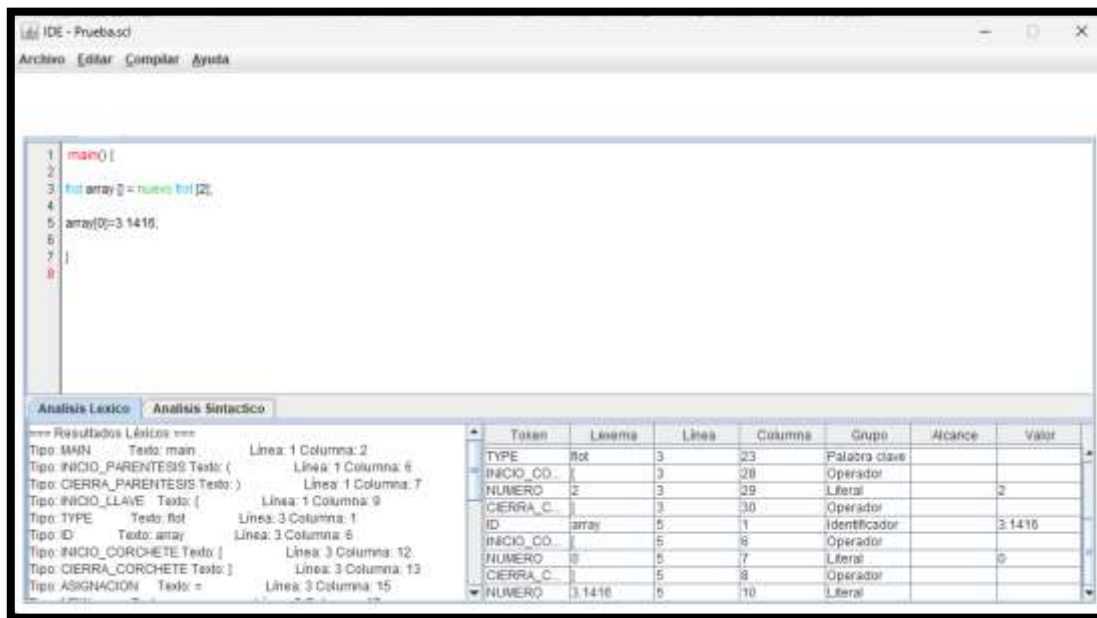
2.4 Pegar

Pega el contenido del portapapeles en la posición actual del cursor.



3.- Compilar

Ejecuta el proceso de compilación del código fuente. El resultado de la compilación se muestra en la parte inferior del IDE. A continuación, se puede observar un ejemplo de instrucciones para un arreglo.



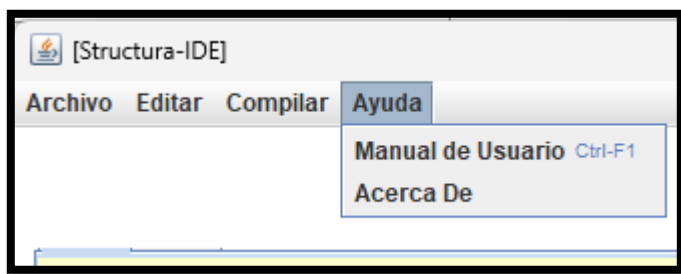
4.- Ayuda

4.1 Manual de usuario

Abre un archivo en formato PDF con instrucciones detalladas sobre el uso del IDE.

4.2 Acerca de

Muestra información sobre la versión y características del entorno de desarrollo integrado.

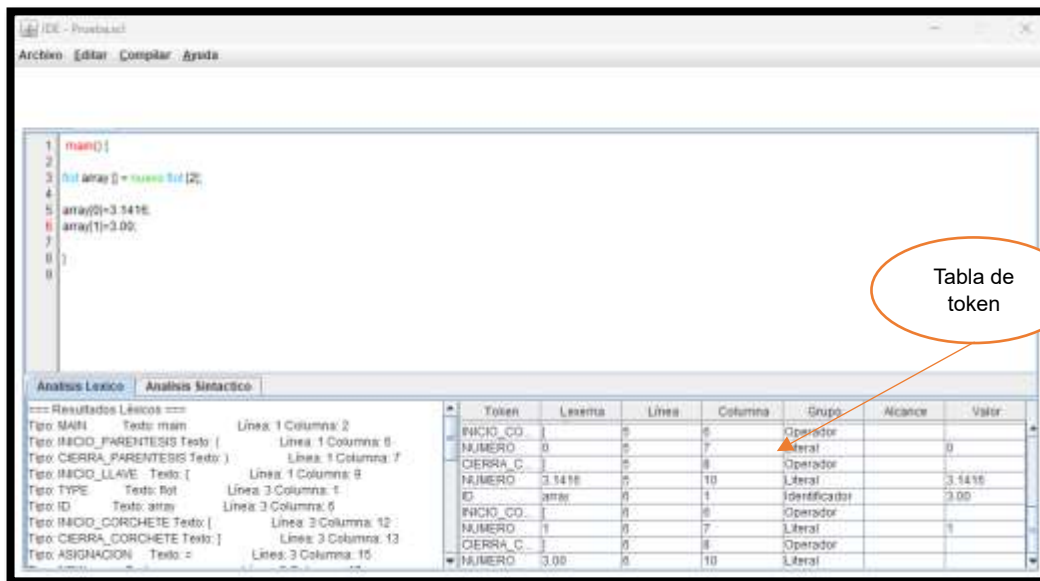


Los submenús se pueden realizar por medio de las combinaciones de teclado, como se muestra en el recuadro.

5._ Ejemplo de compilación

Arreglo (array):

El siguiente código que se ha compilación es correcto, demostrando los resultados del analizador léxico, la tabla de tokens generada. En el analizador sintáctico se muestra el mensaje de compilación y la representación del árbol sintáctico.



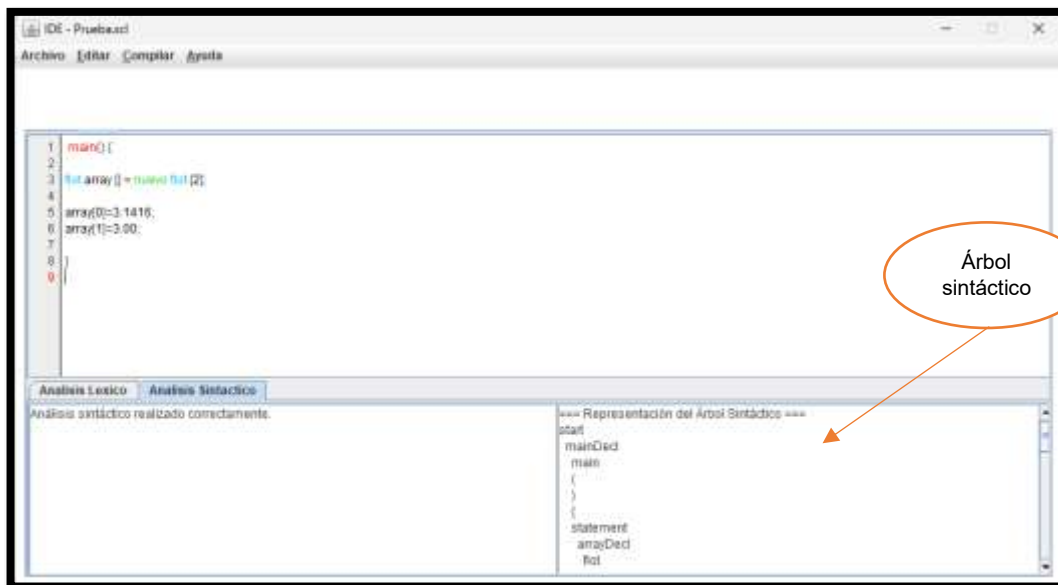
The screenshot shows the IDE interface with the Lexical Analysis results. The code in the editor is:

```
1 main() {  
2  
3 int array[] = {1, 2, 3};  
4  
5 array[0] = 3.1415;  
6 array[1] = 3.00;  
7  
8 }  
9
```

The Lexical Analysis results are displayed in the bottom panel. The Token table is highlighted, showing the following data:

Token	Lexema	Línea	Columna	Grupo	Alcance	Valor
INICIO_CO	{	1	5	Operador		
NUMERO	0	5	7	Literal		0
CIERRA_C	}	5	8	Operador		
NUMERO	3.1415	5	10	Literal		3.1415
ID	array	5	1	Identificador		
INICIO_CO	{	5	5	Operador		
NUMERO	1	5	7	Literal		1
CIERRA_C	}	5	8	Operador		
NUMERO	3.00	5	10	Literal		3.00

Tabla de token

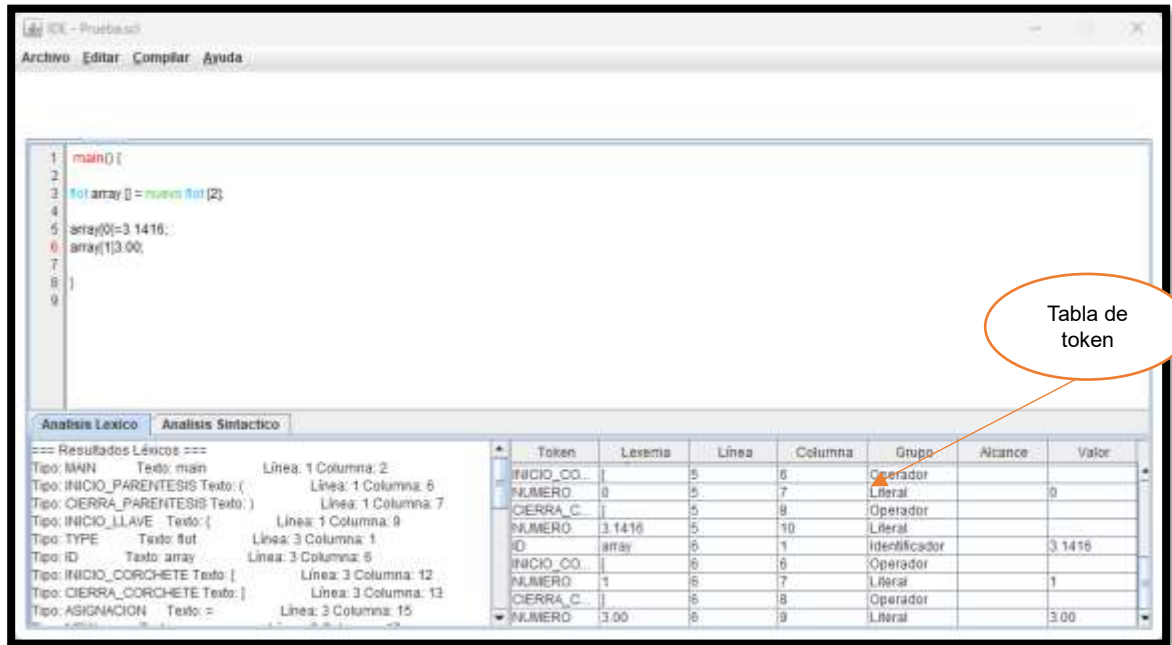


The screenshot shows the IDE interface with the Syntax Analysis results. The code in the editor is the same as in the previous screenshot. The Syntax Analysis results are displayed in the bottom panel. The message "Análisis sintáctico realizado correctamente." is shown. The Syntax tree representation is displayed in the bottom right panel, showing the following structure:

```
start  
mainDecl  
main  
{  
statement  
arrayDecl  
}
```

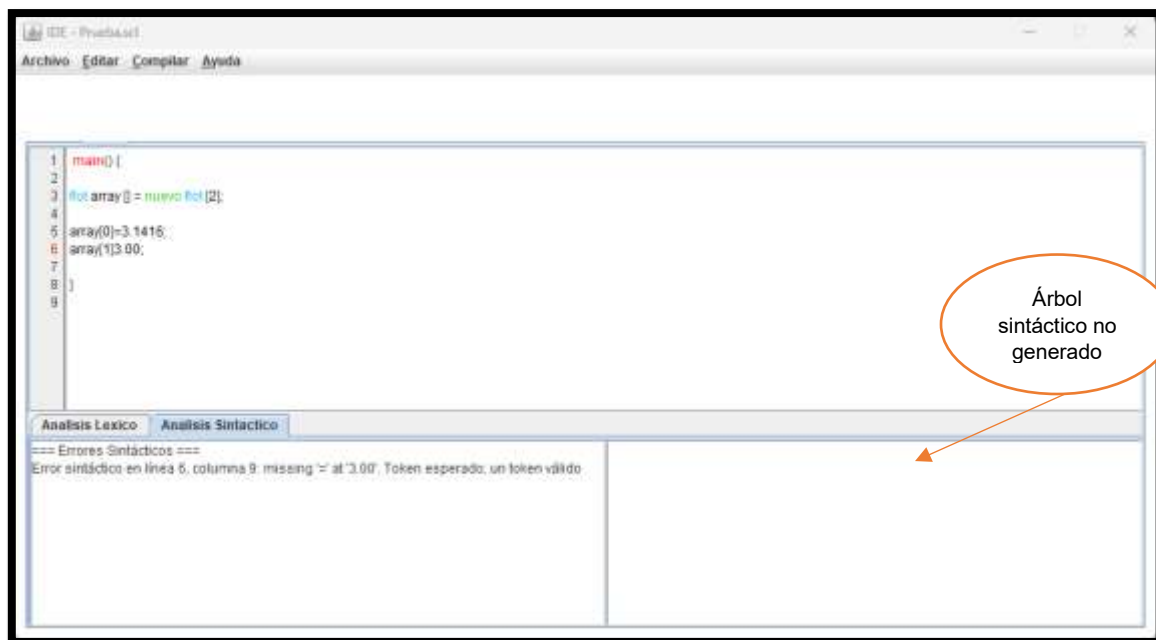
Árbol sintáctico

En dado caso que el analizador sintáctico contenga un error se mostrara cuál es el posible token faltante. En este ejemplo en la línea 6 le falta el token del signo de '='. En el análisis sintáctico, se demuestra el error en la línea 6 y el token esperado. Dado que el analizador sintáctico es incorrecto no se genera el árbol sintáctico.



Analisis Sintactico

Token	Lexema	Línea	Columna	Grupo	Alcance	Valor
INICIO_CO...	{	1	2	Operador		
NUMERO	0	5	7	Líteral		0
CIERRA_C	}	5	8	Operador		
NUMERO	3.1416	5	10	Líteral		
ID	array	6	1	Identificador		3.1416
INICIO_CO...	{	6	6	Operador		
NUMERO	1	6	7	Líteral		1
CIERRA_C	}	6	8	Operador		
NUMERO	3.00	6	9	Líteral		3.00



Analisis Sintactico

Errores Sintácticos

Error sintáctico en línea 6, columna 9: missing '=' at '3.00', Token esperado: un token válido